

Homework 5

Jake Mc Leroth

2025-05-01

```
library(dplyr)
library(MASS)
options(rgl.useNULL = TRUE)
library(rgl)
library(adabag)
library(ggplot2)
library(xgboost)
```

We make use of the following function to generate the data used, consisting of a label y and two input features, x1 and x2.

```
# Data generating function
GenData<-function(){
  I<-diag(1,ncol=2,nrow=2)
  mu1<-mvrnorm(10,c(1,0),I)
  mu2<-mvrnorm(10,c(0,1),I)
  class1<-class2<-matrix(0,ncol=2,nrow=100)
  for(i in 1:100){
    Ind1<-sample(1:10,1,replace=TRUE)
    Ind2<-sample(1:10,1,replace=TRUE)
    class1[i,]<-mvrnorm(1,mu1[Ind1,],I/5)
    class2[i,]<-mvrnorm(1,mu2[Ind2,],I/5)
  }
  Resp<-matrix(c(rep(1,100),rep(2,100)))
  Pred<-rbind(class1,class2)
  Train<-cbind(Resp,Pred)
  colnames(Train)<-c("y","x1","x2")
  Train
}

set.seed(123) # For reproducibility
data <- as.data.frame(GenData())
data$y <- as.factor(data$y) # Labels need to be factors
data_ada <- data # Labels must be 1 or 2 for AdaBoost
data$y <- as.numeric(data$y) - 1 # change labels to 0 and 1
data_xg <- data # Labels must be 0 or 1 for xgboost

# Range of features (min and max of features)
x1_range <- range(data$x1)
x2_range <- range(data$x2)
```

Question 4

For question 4, the AdaBoost.M1 model is fitted on the simulated data.

```
# Fit AdaBoost.M1
ada_fit <- boosting(y ~ ., data = data_ada, boos = TRUE, mfinal = 20 ,
  coeflearn = "Freund", control = rpart.control(maxdepth = 1))
```

Unlike with other models, the decision boundary for AdaBoost can't be expressed as a function. To overcome this, we can visualise it by making predictions over a grid of many possible pairs of (x1,x2) values that are in the range of the features space. To do this, a sequence of equidistant values between the lowest and highest x1 values are generated. The same is done for x2. The expand.grid() function returns a grid of these sequences i.e. every possible combination of (x1,x2) pairs. Each sequence is 200 numbers in length so that gives 40000 pairs. Each pair represents a potential (x1,x2) combination in the feature space. On each of these pairs, a prediction is made using the fitted AdaBoost model. Every point that is predicted as 1 is highlighted blue, and every point predicted as 2 is highlighted red. This results in a colour coded grid, where the change from blue to red is the decision boundary. A dashed line is then added where the probability of class 1 is 0.5, showing the physical decision boundary between the red and blue sections. The actual data is then overlayed on this. The region a point is in shows which class the model predicted it to be, and the actual color of the point shows which class it actually is (i.e. the same colour means the prediction was correct).

```
# ALL combinations of values for grid
grid_ada <- expand.grid(
  x1 = seq(x1_range[1], x1_range[2], length.out = 200),
  x2 = seq(x2_range[1], x2_range[2], length.out = 200)
)

# Get prediction from grid values
grid_pred_ada <- predict.boosting(ada_fit, newdata = grid_ada)
grid_ada$pred <- grid_pred_ada$class
grid_ada$prob_class1 <- grid_pred_ada$prob[,1]

# Colours for grid and data
grid_col_ada <- ifelse(grid_ada$pred == 1, "lightblue", "lightpink")
actual_col_ada <- ifelse(data_ada$y == 1, "blue", "red")

# Plot grid with colours
plot(grid_ada$x1, grid_ada$x2, col = grid_col_ada, pch = 20, cex = 0.4,
  xlab = "x1", ylab = "x2", main = "AdaBoost Decision Boundary")

# Plot points with colours
points(data_ada$x1, data_ada$x2, col = actual_col_ada, pch = 19, cex = 0.8)

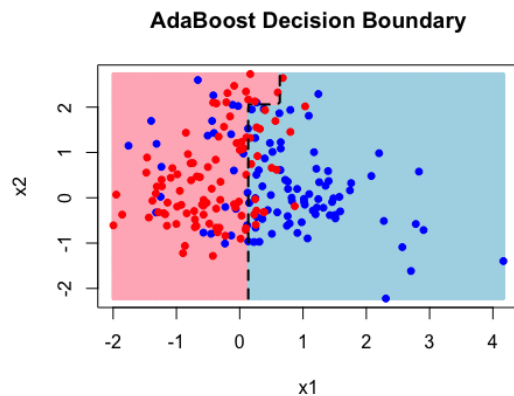
# Get predictions in grid format. The contour line (decision boundary) will
# plotted between grid class predictions
z_matrix <- matrix(grid_ada$prob_class1,
  nrow = length(unique(grid_ada$x1)),
```

```

ncol = length(unique(grid_ada$x2)),
byrow = F)

# Plot decision boundary
contour(x = sort(unique(grid_ada$x1)), y = sort(unique(grid_ada$x2)), z =
z_matrix, levels = 0.5,
drawlabels = FALSE, add = TRUE, lwd = 2, lty = 2, col = "black")

```



Question 5

For question 5, two XGBoost models are fitted, one using the Exponential loss and the other the Hinge loss. For comparison purposes, both used the binary classification error rate as the evaluation metric.

```

# Get data into correct format for XGBoost
xgb <- xgb.DMatrix(data = as.matrix(data_xg[, -1]), label = data_xg$y)

# Params for XGBoost with hinge Loss
params_hinge <- list(booster = "gbtree", eta = 0.5, max_depth = 1, objective
= "binary:hinge", eval_metric = "error")

# Params for XGBoost with exponential
params_exp <- list(booster = "gbtree", eta = 0.5, max_depth = 1, objective =
"binary:logistic", eval_metric = "error")

# Fit XGBoost models
xgb_hinge_fit <- xgb.train(params = params_hinge, data = xgb, nrounds = 100,
verbose = FALSE)
xgb_exp_fit <- xgb.train(params = params_exp, data = xgb, nrounds = 100,
verbose = FALSE)

```

The method of using a grid to plot the decision boundary is used again, comparing the decision boundaries for the two XGBoost models. The decision boundaries for both models are shown:

```

# ALL combinations of values for XGBoost grid
grid_xg <- expand.grid(
  x1 = seq(x1_range[1], x1_range[2], length.out = 200),

```

```

x2 = seq(x2_range[1], x2_range[2], length.out = 200)
)

# Get predictions from grid values for both XGBoost models
grid_pred_xg_exp <- predict(xgb_exp_fit, newdata = as.matrix(grid_xg))
grid_pred_xg_hinge <- predict(xgb_hinge_fit, newdata = as.matrix(grid_xg))
grid_xg$pred_exp <- as.numeric(grid_pred_xg_exp > 0.5)
grid_xg$pred_hinge <- grid_pred_xg_hinge

# Colours for grid and data
grid_col_xg_exp <- ifelse(grid_xg$pred_exp == 0, "lightblue", "lightpink")
grid_col_xg_hinge <- ifelse(grid_xg$pred_hinge == 0, "lightblue",
"lightpink")
actual_col_xg <- ifelse(data_xg$y == 0, "blue", "red")

# Plot grid with colours for Exponential Loss XGBoost
plot(grid_xg$x1, grid_xg$x2, col = grid_col_xg_exp, pch = 20, cex = 0.4,
      xlab = "x1", ylab = "x2", main = "XGBoost (Exponential Loss) Decision
Boundary")

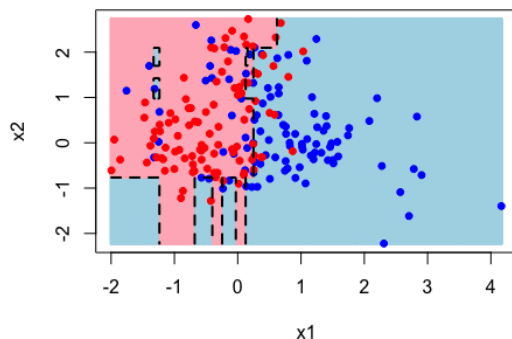
# Plot points with colours for Exponential Loss XGBoost
points(data_xg$x1, data_xg$x2, col = actual_col_xg, pch = 19, cex = 0.8)

# Get predictions in grid format. The contour line (decision boundary) will
plotted between grid class predictions
z_matrix_exp <- matrix(grid_xg$pred_exp,
                       nrow = length(unique(grid_xg$x1)),
                       ncol = length(unique(grid_xg$x2)),
                       byrow = F)

# Plot decision boundary for Exponential Loss XGBoost
contour(x = sort(unique(grid_xg$x1)), y = sort(unique(grid_xg$x2)), z =
z_matrix_exp, levels = 0.5,
        drawlabels = FALSE, add = TRUE, lwd = 2, lty = 2, col = "black")

```

XGBoost (Exponential Loss) Decision Boundary



```

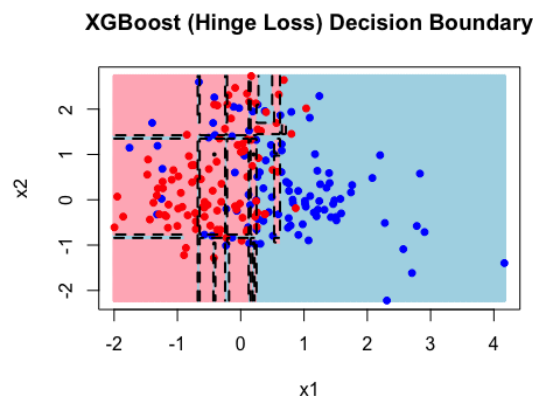
# Plot grid with colours for Hinge Loss XGBoost
plot(grid_xg$x1, grid_xg$x2, col = grid_col_xg_hinge, pch = 20, cex = 0.4,
     xlab = "x1", ylab = "x2", main = "XGBoost (Hinge Loss) Decision
Boundary")

# Plot points with colours for Hinge Loss XGBoost
points(data_xg$x1, data_xg$x2, col = actual_col_xg, pch = 19, cex = 0.8)

# Get predictions in grid format. The contour line (decision boundary) will
# be plotted between grid class predictions
z_matrix_hinge <- matrix(grid_xg$pred_hinge,
                        nrow = length(unique(grid_xg$x1)),
                        ncol = length(unique(grid_xg$x2)),
                        byrow = F)

# Plot decision boundary for Hinge Loss XGBoost
contour(x = sort(unique(grid_xg$x1)), y = sort(unique(grid_xg$x2)), z =
z_matrix_hinge, levels = 0.5,
       drawlabels = FALSE, add = TRUE, lwd = 2, lty = 2, col = "black")

```



As can be seen, the hinge loss results in a much stricter decision boundary. To explore the effect of the learning rate, tree depth and the number of iterations, the following function was created for efficiency. The exponential loss was used for all comparisons.

```

# Function to fit and plot decision boundary of XGBoost models
fit_xg_and_plot <- function(booster = "gbtree", eta = 0.5, max_depth = 1,
objective = "binary:logistic", eval_metric = "error", data, nrounds = 100,
verbose = FALSE, plot_title){
  x1_range <- range(data$x1)
  x2_range <- range(data$x2)
  xgb <- xgb.DMatrix(data = as.matrix(data[, -1]), label = data$y) # data must
# be in correct form already
  params <- list(booster = booster, eta = eta, max_depth = max_depth,
objective = objective, eval_metric = eval_metric)
  xgb_fit <- xgb.train(params = params, data = xgb, nrounds = nrounds,
verbose = verbose)
}

```

```

grid <- expand.grid(
  x1 = seq(x1_range[1], x1_range[2], length.out = 200),
  x2 = seq(x2_range[1], x2_range[2], length.out = 200)
)

grid_pred <- predict(xgb_fit, newdata = as.matrix(grid))
if (objective == "binary:hinge") {
  grid$pred <- grid_pred
} else {
  grid$pred <- as.numeric(grid_pred > 0.5)
}

grid_col <- ifelse(grid$pred == 0, "lightblue", "lightpink")
actual_col <- ifelse(data$y == 0, "blue", "red")

plot(grid$x1, grid$x2, col = grid_col, pch = 20, cex = 0.4,
      xlab = "x1", ylab = "x2", main = plot_title)

points(data$x1, data$x2, col = actual_col, pch = 19, cex = 0.8)

z_matrix <- matrix(grid$pred,
                    nrow = length(unique(grid$x1)),
                    ncol = length(unique(grid$x2)),
                    byrow = F)

contour(x = sort(unique(grid$x1)), y = sort(unique(grid$x2)), z = z_matrix,
        levels = 0.5,
        drawlabels = FALSE, add = TRUE, lwd = 2, lty = 2, col = "black")
}

```

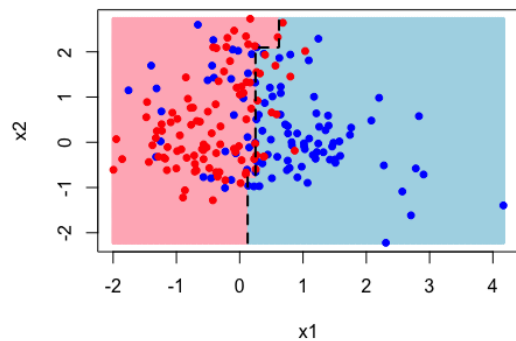
To look at the effect of the learning rate, the decision boundary is shown for models with learning rates from 0.1 to 1:

```

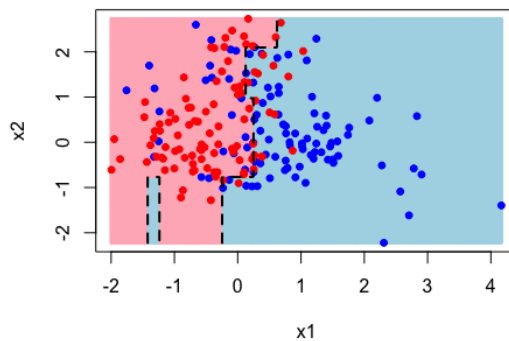
# Loop through different learning rate values
for (eta_val in seq(0.1, 1.0, by = 0.1)) {
  fit_xg_and_plot(eta = eta_val, data = data_xg, plot_title = paste0("XGBoost
(eta = ", eta_val, ") Decision Boundary"))
}

```

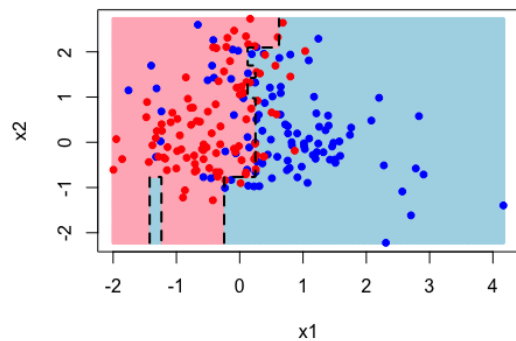
XGBoost (eta = 0.1) Decision Boundary



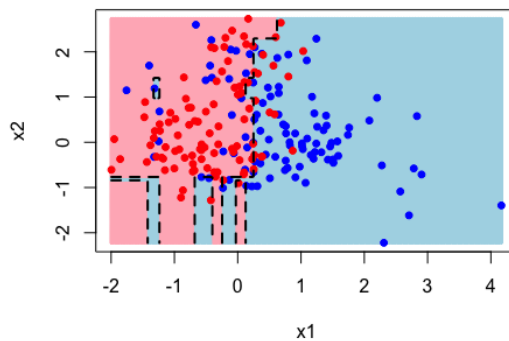
XGBoost (eta = 0.2) Decision Boundary



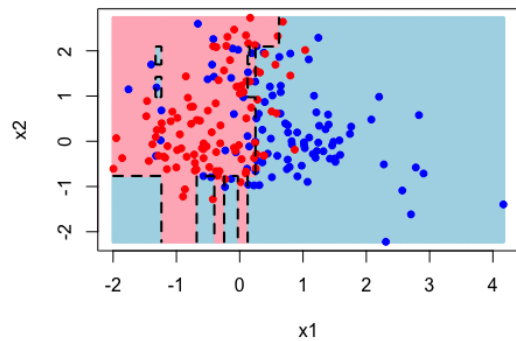
XGBoost (eta = 0.3) Decision Boundary



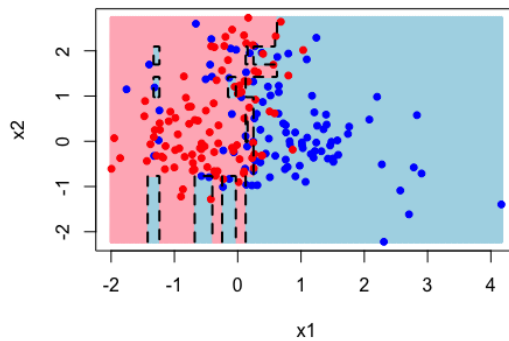
XGBoost (eta = 0.4) Decision Boundary



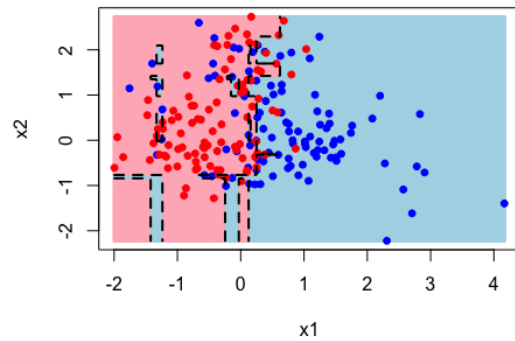
XGBoost (eta = 0.5) Decision Boundary



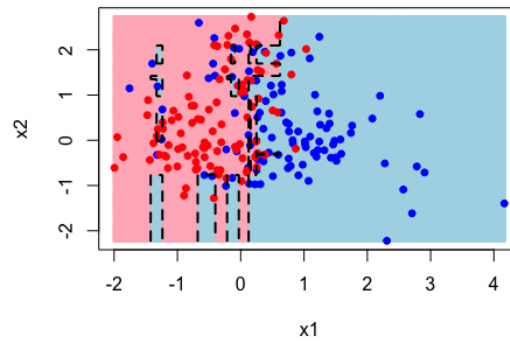
XGBoost (eta = 0.6) Decision Boundary



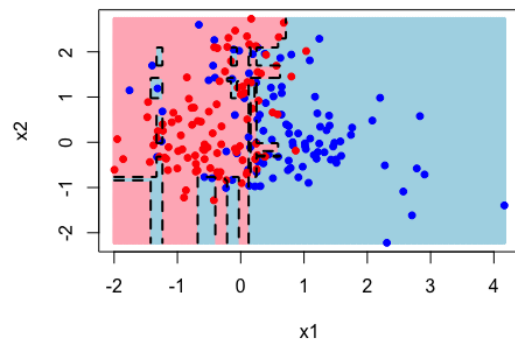
XGBoost (eta = 0.7) Decision Boundary



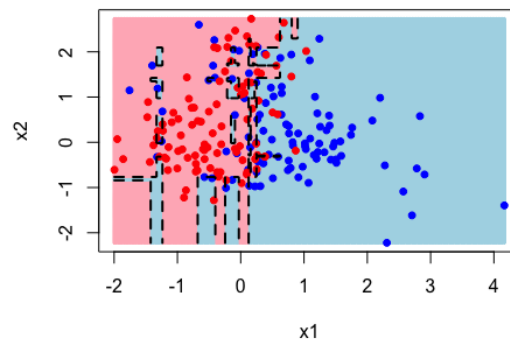
XGBoost (eta = 0.8) Decision Boundary



XGBoost (eta = 0.9) Decision Boundary



XGBoost (eta = 1) Decision Boundary

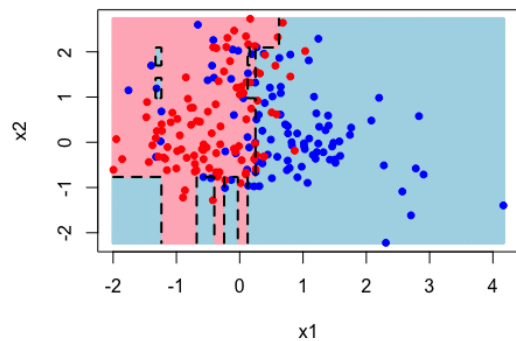


The output shows that as the learning rate increases, the decision boundary gets stricter. After each iteration, the decision boundary changes slightly from the previous, getting stricter each time. The effect of tree depth is looked at next:

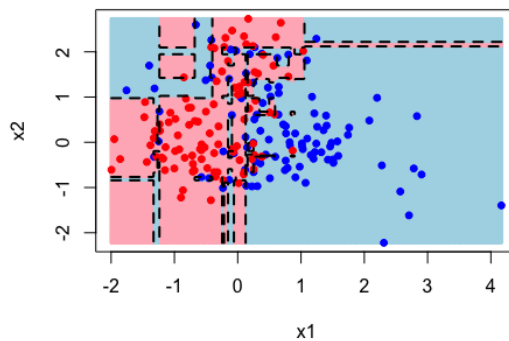
Loop through different tree depth values

```
for (depth in c(1:10, 20, 50, 100)) {  
  fit_xg_and_plot(max_depth = depth, data = data_xg, plot_title =  
    paste0("XGBoost (max_depth = ", depth, ") Decision Boundary"))  
}
```

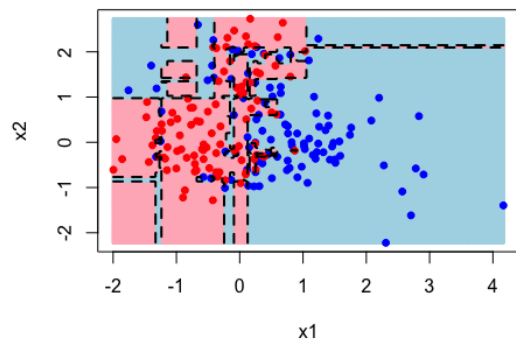

XGBoost (max_depth = 1) Decision Boundary



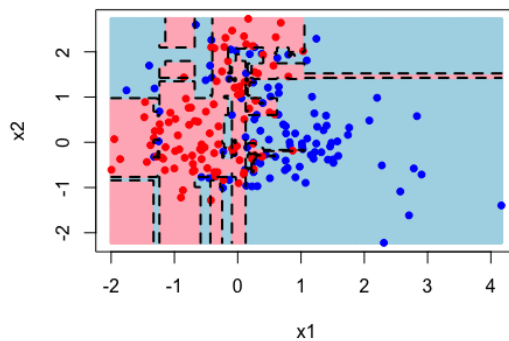
XGBoost (max_depth = 2) Decision Boundary



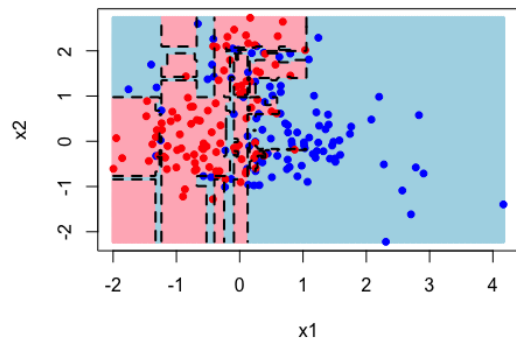
XGBoost (max_depth = 3) Decision Boundary



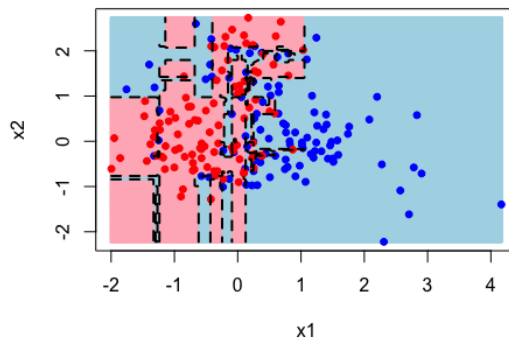
XGBoost (max_depth = 4) Decision Boundary



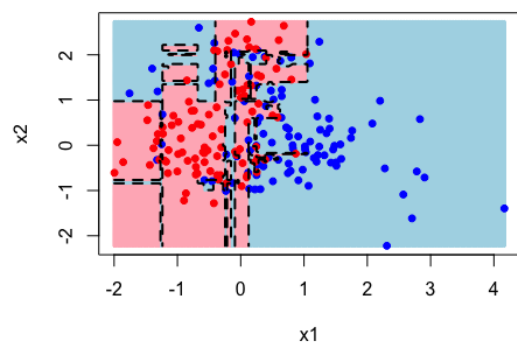
XGBoost (max_depth = 5) Decision Boundary



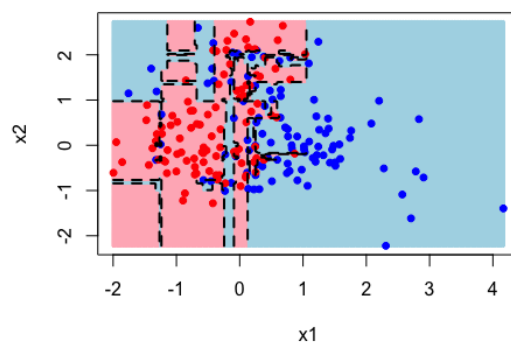
XGBoost (max_depth = 6) Decision Boundary



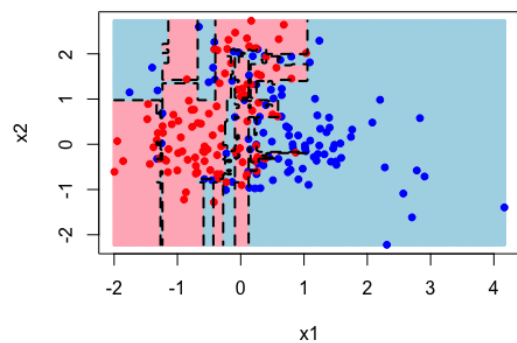
XGBoost (max_depth = 7) Decision Boundary



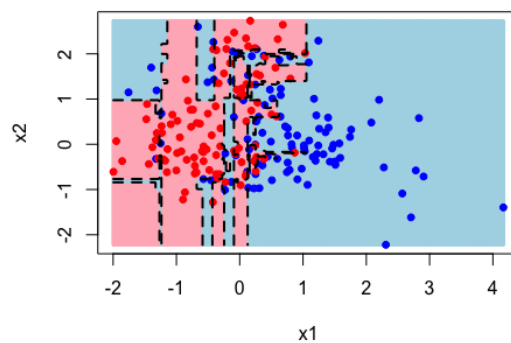
XGBoost (max_depth = 8) Decision Boundary



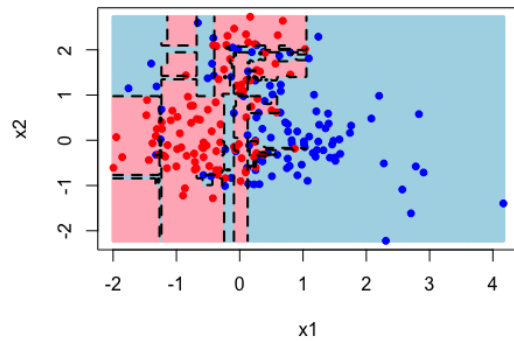
XGBoost (max_depth = 9) Decision Boundary



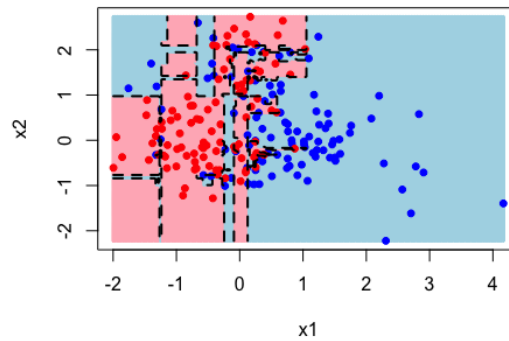
XGBoost (max_depth = 10) Decision Boundary



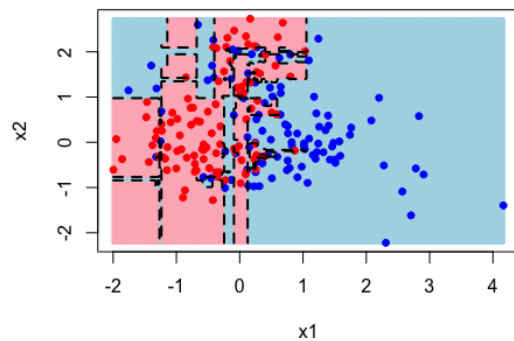
XGBoost (max_depth = 20) Decision Boundary



XGBoost (max_depth = 50) Decision Boundary



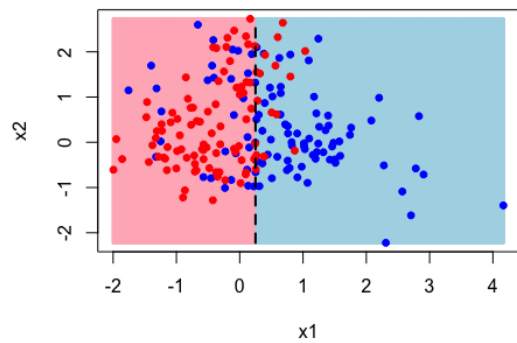
XGBoost (max_depth = 100) Decision Boundary



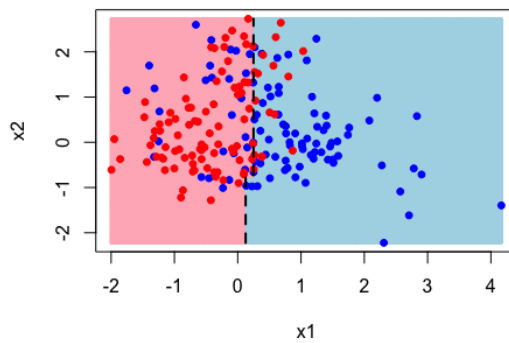
The output shows that, again, the decision boundary gets stricter as tree depth increases. It happens in a different manner to the learning rate, however. Each iteration of the learning rate only changed the existing decision boundary slightly, whereas the changes in tree length often results in very different looking decision boundaries. The decision boundary converges at depth of 20, with each one after that being the same. Finally, the effect of the number of iterations is looked at next:

```
# Loop through different iteration sizes
for (nround in c(1,10, 20, 50, 100, 200, 500, 1000)) {
  fit_xg_and_plot(nrounds = nround, data = data_xg, plot_title =
paste0("XGBoost (nrounds = ", nround, ") Decision Boundary"))
}
```

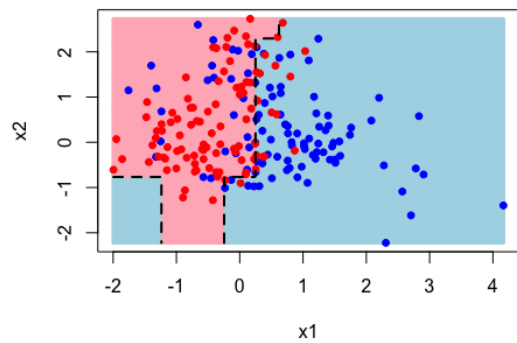
XGBoost (nrounds = 1) Decision Boundary



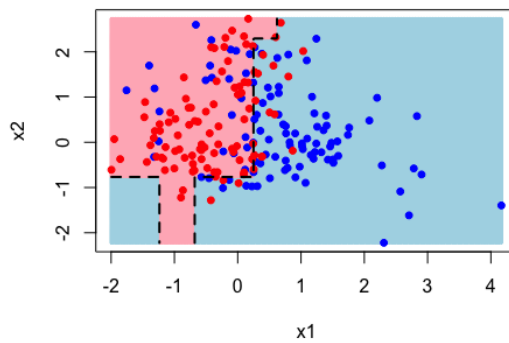
XGBoost (nrounds = 10) Decision Boundary



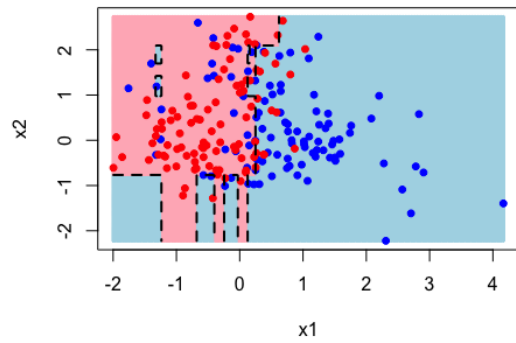
XGBoost (nrounds = 20) Decision Boundary



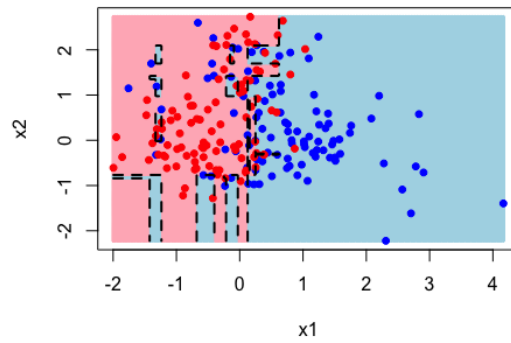
XGBoost (nrounds = 50) Decision Boundary



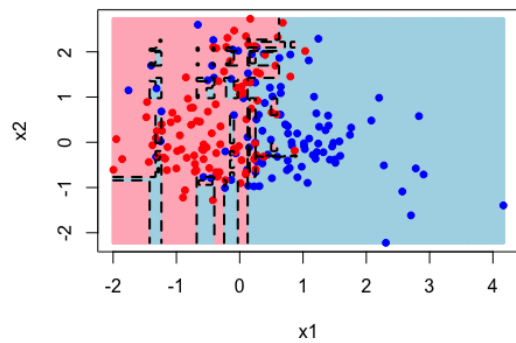
XGBoost (nrounds = 100) Decision Boundary



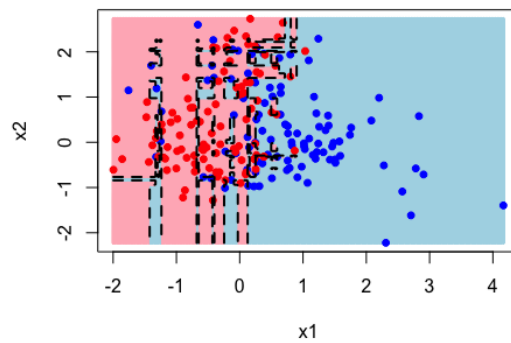
XGBoost (nrounds = 200) Decision Boundary



XGBoost (nrounds = 500) Decision Boundary



XGBoost (nrounds = 1000) Decision Boundary



Again, the decision boundary gets stricter. It can be seen that as it gets sticter, it is just adding a new section to the original decision boundary.